

TECHNOLOGY DECISION RECORD

Why We Built Sales Eyes This Way

A business-oriented look at the technology choices behind Sales Eyes — the problem each choice solves, the alternatives we weighed, and why the chosen stack wins on cost, speed, and scalability.

Build vs. Buy

Cost Efficiency

Speed to Market

Scalability

1

The Business Problem

WHY THIS PROJECT EXISTS

Sales reps lose deals not because they lack a good pitch, but because they walk into conversations without enough context about who they're talking to. Manually researching a prospect — their company, their role, recent news, likely pain points — is slow, inconsistent between reps, and easy to skip when the pipeline is busy.

Sales Eyes was built to close that gap: turn a few minutes of typing into a structured, AI-assisted research profile and a ready-to-use, methodology-driven sales script — without requiring a dedicated research or sales-ops team.

~30 min

Manual research time saved per prospect

\$0

Paid search/API cost at MVP stage

1

Command to stand up the entire stack

Every technology decision below was made against the same constraint set: this needed to be built and validated **fast**, by a **small team**, at **low infrastructure cost**, while staying **credible enough to scale** if the product proves out. That trade-off — speed and cost now, without boxing in the future — is the thread running through every choice in this document.

2

Decision Criteria

HOW WE EVALUATED EVERY OPTION

1

Speed to Build
Can a small team ship a working version in weeks, not quarters?

2

Cost at Low Scale
Does it stay cheap (or free) while the product is unproven?

3

Talent Availability
Is it easy to hire for, or does it lock us into scarce specialists?

4

Ecosystem & Support
Are there mature libraries and a large community to lean on?

5

Room to Scale
Will it still make sense if usage grows 10-100x?

6

Vendor Flexibility
Can we swap providers later without a rewrite?

Reading this document: each section that follows names the business problem in that category, what we chose, what we considered instead, and why the trade-off favored the chosen option today.

FRONTEND FRAMEWORK

Next.js + React + TypeScript

Business problem: we needed a fast, professional-feeling web app that reps could use from any browser, built by developers who are easy to find and onboard, without spending weeks on plumbing like routing and page structure.

Fastest path to a working UI

File-based routing and built-in tooling mean new screens ship in hours, not days.

Largest talent pool

React is the most widely known frontend library, keeping hiring and onboarding cheap.

Type safety reduces bugs

TypeScript catches data-shape mistakes (e.g. a missing field from the API) before they reach a user.

SEO & performance ready

Next.js supports server rendering out of the box, useful if a public-facing site is added later.

Option	Advantage	Why not chosen
Next.js (chosen)	Batteries-included React framework; fast setup; huge ecosystem	—
Plain React (Vite/CRA)	Slightly lighter weight for a pure single-page app	We'd hand-build routing, SSR, and conventions Next.js already provides
Vue.js	Gentle learning curve, good documentation	Smaller hiring pool and third-party component ecosystem than React
Angular	Very structured, good for large enterprise teams	Heavier framework and steeper learning curve than an MVP needs

Business problem: we needed a consistent, professional look without hiring a dedicated designer or writing a custom CSS system from scratch.

Design consistency for free

A shared set of spacing/color/typography rules keeps every screen looking coherent.

No context-switching

Styles live next to the markup, so developers move faster without hopping between files.

Option	Advantage	Why not chosen
Tailwind CSS (chosen)	Fast to write, consistent design tokens, tiny production bundle	—
Plain CSS / Sass	Full control, no framework opinions	Slower to write consistently; easy to drift into inconsistent styling
Component library (MUI, Chakra)	Pre-built polished components	Harder to get a distinctive brand look; more dependency weight

4

Backend Layer

THE ENGINE DOING THE RESEARCH AND REASONING

BACKEND FRAMEWORK & LANGUAGE

FastAPI (Python)

Business problem: the core value of this product — AI orchestration, web scraping, data extraction — lives in Python's ecosystem. We needed a backend framework that plays to that strength while staying fast and lightweight to build with.

Best AI/data ecosystem

Nearly every AI SDK, scraping, and NLP library targets Python first.

Async by default

Handles many simultaneous research jobs (web calls, AI calls) without blocking.

Self-documenting API

Automatic interactive docs (Swagger) speed up frontend/backend collaboration.

Built-in data validation

Pydantic schemas catch malformed data before it hits the database or the AI.

Option	Advantage	Why not chosen
FastAPI (chosen)	Async, fast, auto-docs, ideal for AI/data-heavy workloads	—
Django	"Batteries-included," strong admin panel, mature ORM	Heavier and more opinionated than a lean AI-orchestration API needs
Node.js / Express	Same language as the frontend (JavaScript)	Weaker ecosystem for the AI/scraping/data-extraction work this app is built on
Java / Spring Boot	Very mature for large enterprise systems	Slower to build and iterate with; overkill for a fast-moving MVP

DATABASE ACCESS

SQLAlchemy (async) + Alembic

Business problem: the data model (users, sessions, findings, intelligence) would keep evolving as features were added, and we needed to track those changes safely without hand-writing risky SQL migrations.

Safer schema changes

Alembic version-controls every database change, so upgrades are repeatable and reversible.

Async performance

Non-blocking database calls keep the API responsive under concurrent research jobs.

Bottom line: Python + FastAPI let one small team own the entire AI research pipeline — scraping, extraction, and orchestration — without stitching together services in multiple languages.

PRIMARY DATABASE

PostgreSQL

Business problem: the data here is fundamentally relational — users own sessions, sessions own findings, findings connect to candidates and conflicts. We needed strong guarantees that this data stays consistent and queryable as relationships grow.

Data integrity
Enforced relationships prevent orphaned or inconsistent records — important when facts are being cross-referenced.

Flexible enough for AI data
Native JSON support handles semi-structured findings without needing a second database.

Free and open source
No licensing cost at any scale, with enterprise-grade reliability.

Easy path to managed hosting
Every major cloud provider offers managed Postgres, simplifying a future production move.

Option	Advantage	Why not chosen
PostgreSQL (chosen)	Relational integrity + JSON flexibility, free, widely supported	—
MySQL	Also free, very common	Weaker native JSON/semi-structured data support than Postgres
MongoDB	Flexible schema, fast to prototype	Our data is highly relational; we'd fight the document model constantly
SQLite	Zero setup, great for local testing	Not built for concurrent multi-user production traffic

AI PROVIDER

DeepSeek (OpenAI-compatible API)

Business problem: the product's core cost driver at scale is AI usage — every research plan and every script is an AI call. We needed strong reasoning quality at a price that doesn't erode margins as usage grows, without rewriting the app if we switch providers later.

Significantly lower cost per call

Materially cheaper than premium-tier models, which matters directly to unit economics at scale.

Strong reasoning for the task

Capable enough for plan generation, extraction, and script writing — the jobs this app actually needs.

Provider portability

Built against an OpenAI-compatible API shape, so swapping providers is a config change, not a rewrite.

Option	Advantage	Why not chosen (today)
DeepSeek (chosen)	Strong reasoning at a fraction of premium-model pricing	—
OpenAI (GPT)	Best-in-class reasoning, huge ecosystem	Meaningfully higher per-call cost at this stage of the product
Anthropic (Claude)	Excellent reasoning and reliability	Premium pricing tier; kept as a future upgrade path if margins allow
Open-source self-hosted (Llama)	No per-call fees	Requires GPU infrastructure and ops overhead we don't need yet

Note on flexibility: because the integration follows the OpenAI-compatible request format, moving to a higher-tier model later — if quality demands justify the cost — is a low-risk change, not a re-architecture.

7

Research & Data Sourcing

WHERE THE RAW FACTS COME FROM

WEB SEARCH PROVIDER

DuckDuckGo Search

Business problem: every research session needs multiple web searches. At MVP stage, paid search APIs (Google/Bing) would add a real per-query cost before the product had proven it drives revenue.

Zero cost, no API key

Removes a recurring expense and a signup/billing dependency during validation.

Fast to integrate

No approval process or quota management to slow down early development.

Option	Advantage	Why not chosen (today)
DuckDuckGo (chosen)	Free, no key required, good enough coverage for MVP research	—
Google Custom Search API	Best result quality and coverage	Per-query pricing; adds cost before the product is validated
Bing Search API	Strong quality, enterprise SLAs	Also metered/paid; a candidate for a later scale-up

- Cost: \$0 today
- Swap path: config-level change
- Trade-off: less coverage than paid search

PAGE EXTRACTION

BeautifulSoup + lxml

Business problem: once a page is found, we need to reliably pull structured facts out of messy, inconsistent HTML from many different company websites.

Battle-tested reliability

Handles malformed HTML gracefully, which is the norm on the open web, not the exception.

No licensing cost

Fully open source, keeping the research pipeline's marginal cost at zero.

DEPLOYMENT MODEL

Docker Compose

Business problem: the team needed the entire stack (frontend, backend, database) to run identically on any developer's laptop and, later, on a server — without hiring dedicated DevOps early on.

One command to run everything
New developers are productive within minutes, cutting onboarding cost.

Environment parity
What runs locally matches what runs in production, reducing "works on my machine" issues.

Low operational overhead
No cluster to manage at this stage of the product.

Clear upgrade path
Containers built for Compose move to Kubernetes or a managed container service with minimal rework if scale demands it.

Option	Advantage	Why not chosen (today)
Docker Compose (chosen)	Simple, low-overhead, consistent environments	—
Kubernetes	Industry standard for large-scale orchestration	Operational complexity and cost not justified pre-product-market-fit
Serverless (Lambda, Cloud Run)	Pay-per-use, auto-scaling	Adds cold-start and state-management complexity to a stateful research pipeline

9

Stack Summary

THE FULL PICTURE, AT A GLANCE

Category	Chosen Technology	Primary Business Reason
Frontend Framework	Next.js + React + TypeScript	Fastest build speed, largest talent pool, fewer runtime bugs
Styling	Tailwind CSS	Consistent design without a dedicated designer
Backend Framework	FastAPI (Python)	Best ecosystem for AI/data work; async performance
Database	PostgreSQL	Strong data integrity for relational, cross-referenced research data
AI Provider	DeepSeek	Strong reasoning at a fraction of premium-tier cost
Web Search	DuckDuckGo Search	Zero marginal cost during product validation
Page Extraction	BeautifulSoup + lxml	Reliable, free, industry-standard scraping tools
Deployment	Docker Compose	Fast onboarding, environment parity, low operational cost

10

Looking Ahead

WHAT WOULD TRIGGER A RE-EVALUATION

These choices were optimized for validating the product quickly and cheaply. As Sales Eyes gains traction, a few decisions are worth revisiting on purpose — not because they were wrong, but because the constraints that shaped them will have changed:

Trigger	Likely Re-evaluation
Script quality becomes a competitive differentiator	Upgrade from DeepSeek to a premium-tier model (Claude/GPT) for specific high-value steps
Research accuracy/coverage complaints increase	Move from DuckDuckGo to a paid search API for broader, higher-quality results
Traffic or team size grows significantly	Move from Docker Compose to a managed container platform or Kubernetes
Multiple regions or heavy concurrent load	Introduce caching, read replicas, or a managed Postgres tier

Why this matters to the business: none of today's choices are dead ends. Every one was picked so that "grow into a bigger version" is a configuration or infrastructure change — not a rewrite of the product.